

FRED TALKS

22nd September 2026

CONTENTS

Ways to think about token pricing

BENEDICT EVANS · 2607 WORDS

Building agents that reach production systems with MCP

CLAUDE.COM · 1746 WORDS

Headless everything for personal AI

INTERCONNECTED · 1335 WORDS

Some new agentic patterns

MASSIVELY PARALLEL PROCRASTINATION · 1776 WORDS

Ways to think about token pricing

BENEDICT EVANS · 09 AUG 2026 · [SOURCE](#)

AI is in a supply crunch today, but what happens when we come out of it? How and where will supply, demand, price, capacity and capex get back into equilibrium? Today, model labs can name their price, but why won't they end up as low-margin commodity infrastructure?

There are only two things you can say with certainty about token prices: we're in a supply crunch, and this is unstable. All of the variables are in play, and the market will get shaken out over the next few years to arrive at a new equilibrium. Right now we have a lot of frantic analysis of 'time to power', but question at the end of that is whether the foundation models have sustainable pricing power, strategic leverage and value capture, or whether they become low-margin commodity infrastructure providers. At the moment, I think every dynamic we can see points to the latter.

Clearly, the situation today is transitory. On the supply side, a trillion dollars or more of data centre capex is coming down the pipe (and plenty more [semiconductor capex](#) behind that), inference efficiency continues to improve very quickly, and new models are far more (or far less!) efficient in their token use. On the demand side, although the market has been [capacity-constrained since 2022](#), the crunch in the first half of this year has been driven by sudden product-market fit in really just one use case, software development, and that's actually a pretty small field (imagine if we had product-market fit for a consumer use case with hundreds of millions of DAUs - today's infrastructure couldn't support it at any price). We don't know what the next use-cases to scale will be, nor when that would be, nor what their token needs would be.

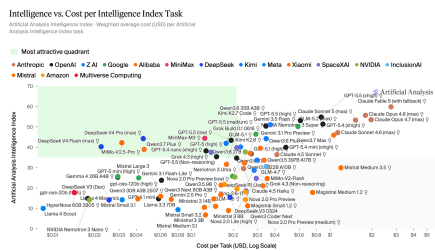
Going up a level, it's been pretty widely reported that inference today has 40-50% gross margins: this includes depreciation of the associated server costs (or the cost of renting them), but we don't really know the asset life (five years? Seven years?) and obviously this doesn't include the cost of training the next model a couple of times a year, which is currently far larger than revenue. In principle, inference is a marginal cost and training is a fixed cost, so with high enough revenue you can reach profitability, but we don't know how training costs will change. On the other side of the table, it's unclear how much of the surge in use in the last few months has an ROI (or at least has an ROI that can be quantified to a CFO), let alone any future use cases, and hence what prices people might be prepared to pay for them.

So, all the variables will move all over the place over the next 12 months, and move again over the next three to five years. How could we suggest where this will settle? How and where will supply, demand, price, capacity and capex get back into equilibrium?

In theory, you can model this bottom-up. You can make some assumptions around each of the variables I suggested above, and then try to model out how many chips there are now, how many chips with what performance TSMC and the rest of the semis industry might be able to deliver when, and how fast all of that can be brought online in data centers and how fast those can be powered. Then you can wonder about price discipline, and make some guesses about use cases. This will get you a number, but it will be rather like trying to build a five-year forecast for the broadband market in 1998: the spreadsheet will be very pretty, and you might even get close to the right number for this year, but there are too many unknown variables to make a useful forecast of a longer-term market structure.

In other words, we can say that token price is a function of supply and demand at a level between the sellers’ marginal cost and the buyers’ ROI, but we don’t actually know what supply, demand, marginal cost or ROI will be.

The other approach is to look from the top down: how do things like this tend to play out? What are the building blocks, and where can they go? Most of this conversation depends on what happens to this curve.



First, how many people will pay to be at the top right of the curve - to be at the frontier? At one extreme there are already use cases that already work just fine with a small, old, perhaps open source model that runs for ‘free’ on-prem or on your phone; at the other extreme there will be some that get better results from the latest, most expensive frontier model, consuming lots of tokens for lots of money; and then there will be many that are somewhere in between. So, how many use cases get better results from going how far up the cost curve, and how many have an ROI for that, and how much of the use is handled by models that are smaller, cheaper, good ‘enough’ and much more commoditised? The Panglossian view is that ROI might go *up* with more expensive frontier models because they have better results, but where does that really apply?

Second, does the frontier keep moving significantly? This is obviously the most basic science question in AI: how long does the frontier keep getting better, how long does that keep needing more and more compute, and does that continue to happen at a rate that keeps it ahead of downward pricing pressure from efficiency and capacity gains? Does the expensive head of the curve continue to be a thing?

Third, will there still be fierce competition between frontier models? Does the field shrink to fewer and fewer frontier models, perhaps with network effects emerging? Do frontier models diverge, with different models having much clearer leads in different fields? That could be another path to sustainable pricing power. Or do we continue with a mid-single-digit number of companies that are all making frontier models that all have generally equivalent capabilities? At the moment, everyone is using mostly the same science and mostly the same training data, and getting mostly the same results, and we don't yet know of a network effect or any other winner-takes-all effect that would let one company pull ahead, stay ahead, and do things that others could not, in some sustainable way. Does that change?

Fourth, how much of the value from those high-end use cases is captured by the frontier model itself? How much needs to be wrapped in tooling, process, proprietary data, go-to-market, networks, support, and everything else associated with a traditional software company, even if you do need the big expensive frontier model underneath? Can that model do the whole thing, or is the model, no matter how good, still a piece of infrastructure that you use to make the actual product? At the extreme, can the model itself invent and make all of those things, and would that let them charge by seat, by outcome or just take the profit? Or do even (or especially) the most sophisticated and high-value use-cases need to sit inside hundreds of new companies that can pick and choose which models to use?

None of these are binaries: they're all a question of degree, and they'll probably vary quite a lot by use case. But at one extreme, there are two or three giant minds that run half of everything and have massive pricing power, and at the other extreme LLMs look like databases - there'll be millions of them, some very big and some very small, and the value is in what you build on top - after all, every SaaS company is a 'database wrapper'. There's a future in which Anthropic (or a company we haven't heard of yet) wins the whole thing and can set its own terms, and a future in which dozens of routers run real-time auctions to allocate your tasks across hundreds of low-margin model-farms and a benchmark company takes a fee on every single one.

I don't think anyone can actually *know* the answer yet. I've said "we don't know" a lot, and that's very deliberate. Part of the concept of the 'S Curve' is that there's a stage early in the emergence of a new technology where it's clear that this is going to be huge but nothing else is clear at all - the mid 1990s for the internet, say, and 2008 or 2009 for mobile. There are places where you can take a view - for example, I've argued at length that I think chatbots are a poor

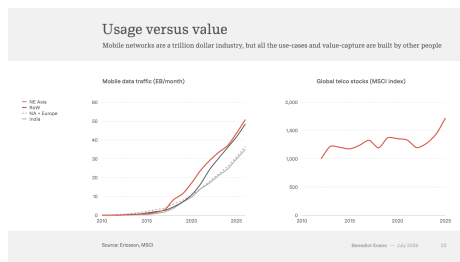
interface that will struggle to capture value up the stack - but we should presume there are big questions that we can't see yet, let alone answer, and anyone picking one of the ten possible outcomes we can see and saying "it will be that one!" is just guessing.

Meanwhile, there is structural uncertainty at the early stages of every big new technology, but the uncertainty now is different, because we don't have a good theoretical understanding of why these models work so well and so we don't know how much better they can get. In 1995, we didn't know how the internet would evolve but we knew that there were less than 100m PCs on earth (and they were expensive) and that telcos couldn't give everyone FTTH next year; in 2010 we didn't know what the next iPhone would be but we knew it wouldn't have retinal projection. We knew the physical limits in ways we don't really know with LLMs. Next month a new approach could cut inference compute needs by 90%, or double demand, or both.

All of this has people hunting for patterns to recognise (indeed, it's been observed that *all* conversations about AI end in a hunt for metaphors). It has become common to draw comparisons with fiber, which had a massive overbuild in the Dotcom bubble that looks a bit like the infrastructure build-out today. The narrow problem with that is that fiber construction was far ahead of demand, where AI compute construction is far behind demand, though, as above, we don't know what that supply/demand balance will look like in the future. But the more relevant objection, I think, is that fiber construction was mostly fixed cost (digging holes) rather than marginal cost (more equipment), whereas growth in compute needs means you need to buy more compute.

That makes mobile data a more fruitful comparison here. Mobile networks have marginal cost for capacity, and like AI they had an enormous surge in usage 15 years ago, that overwhelmed capacity and had carriers scrambling to add capacity and rebalance their pricing. Meanwhile, selling bits looks superficially similar to selling tokens: it's an opaque measure of marginal cost that doesn't map in any transparent or intuitive way to use cases or value, and needs to be replaced with bundles of some kind. But most importantly, in the last 20 years cellular data traffic has risen by several orders of magnitude, and this has become an enormous industry, with annual revenue of a trillion dollars and capex of \$200 billion, but the stocks have gone nowhere, and all the value was captured by other people further up the stack. This, of course, is one of the core questions for AI: is this going to be low-margin commodity infrastructure with all the value captured by other people further up the stack?

[View fullsize](#)



Semiconductor manufacturing also has echoes of AI, since unlike cellular it has something of the same escalating cost and complexity that we see in foundation models. Rock's Law pointed out that the cost of a cutting edge fab doubles every four years: over time the frontier of semiconductors become so hard and so expensive that the number of players dropped from dozens to a handful and now really only one, TSMC. That's another core question for AI: will this get so hard and so expensive that only a couple of people can do it, even without network effects? Equally, many semiconductor uses are further back along the same kind of price/performance curve I just discussed for AI. Even here, though, while TSMC has a *de facto* monopoly on the frontier, and nice margins, it doesn't actually capture a large share of value from the broader tech economy: net income last year was \$53bn, less than half of Apple alone.

There are plenty of other comparisons one could make here: in the last six months Sam Altman has compared OpenAI both to Windows, a high-margin capital-light monopoly based on network effects, and electricity utilities, which are natural monopolies but also low-margin regulated utilities selling a pure commodity. One could also point to cloud (three leading players with good margins and clearly distinguished propositions, but again limited value-capture). But none of these have predictive value: analogies don't have predictive value. You can't prove whether something will have the same outcome as mobile by arguing how much it's like mobile: this was a mistake that many very clever people in tech made 15 years ago, claiming that Android would beat iOS because Android was 'open' and 'open' Wintel had beaten the 'closed' Mac in the 1990s (it's also the mistake that doomers make when they claim that AI is 'like' nuclear weapons). Everything is different: bits, tokens and transistors are different, each of these examples are different from each other, and AI will be different too.

However, these examples do tell us, empirically, that something can be very important, very expensive, change the world, and be full of very sophisticated science and engineering, and yet have a wide range of possible outcomes. There isn't one inevitable path here: you can have price equilibrium at high margins and at low margins, and with and without market concentration, and you can't hand-wave that away by talking about AGI and saying "you don't understand exponentials!"

However, if one thread in everything I've written above is how much we don't yet know, the other thread is that every path to foundation models having market dominance, strategic leverage, value capture, winner-takes-all effects, or anything else other than becoming commodity infrastructure, requires something to change.

Maybe frontier models will become less competitive - yet in the last six months, Mark Zuckerberg and [Elon Musk](#) jumped from zero back onto the leaderboards. Maybe network effects will emerge. Maybe chatbots can grow into products and don't need to be wrapped in software. Maybe one lab will start out-executing all the others and pull ahead on sheer product dynamism - Microsoft, Google, Facebook and Apple had to execute their way into a winning position before they had winner-takes-all efforts. Maybe *something* else will happen.

In particular, we have not one but two potential *dei ex machina* - Trump and China. China is reportedly [considering](#) regulating open source and some people close to Trump have floated this as well (though since Meta abandoned Llama the US has no leading open model), and export controls could expand and become systematic. Many people see the pleas for regulation from Anthropic and (sometimes) OpenAI as a front for regulatory capture, but either way, we can't presume this will remain an entirely free market.

Even so, that brings me back to the same point: the current market dynamics point to a future in which, as today's supply crunch eases, frontier models move towards becoming commodity infrastructure, with all of the value built on top, and for a different outcome, something needs to happen that we don't see yet.

Building agents that reach production systems with MCP

CLAUDE.COM · 23 APR 2026 · [SOURCE](#)

Agents are only as useful as the systems they can reach. Teams tend to converge on three approaches for connecting them to external systems—direct API calls, CLIs, and MCP. This post lays out where each fits, why production agents tend to land on MCP, and the patterns for building those integrations effectively.

Connecting agents to external systems

We generally see three paths for connecting agents to external systems: direct API calls, CLIs, and MCP. Each makes sense somewhere, depending on what you're building. The key distinction is whether there's a common layer between agents and services, and how far that layer reaches.

Direct API calls

The agent calls your API directly—either by writing code that issues HTTP requests inside a code-execution sandbox, or through a generic function-calling tool. This is where most teams start, and it works fine for one agent talking to one service, or a small number of integrations that don't need to be reused across agent platforms.

The challenges start to hit at scale. With no common layer between agents and services, each agent–service pair becomes a bespoke integration with its own auth handling, tool descriptions, and edge cases—the $M \times N$ integration problem.

Command-line interface (CLI)

The agent runs your command-line tool in a shell. This is fast, lightweight, and leans on pre-existing tooling. It works great for local environments and sandboxed containers—anywhere there's a filesystem and a shell. This provides a common layer, but it's thin.

CLIs hit hard limits reaching mobile, web, or cloud-hosted platforms that don't expose a container, and auth is handled by the CLI's own mechanism—usually a credential file on disk. This is best suited to quick, permissive integrations in local environments.

Model Context Protocol (MCP)

MCP provides the common layer as a protocol. The agent connects to a server that exposes your system's capabilities, with auth, discovery, and rich semantics standardized. One remote server reaches any compatible client (Claude, ChatGPT, Cursor, VS Code, and more), in any deployment environment.

It requires a little bit more upfront investment. The return is that the integration is portable, and provides the semantics needed for a feature-rich agent integration.

Production agents run in the cloud

Production agents increasingly run in the cloud, so they can scale and operate continuously. The systems they need to reach are cloud-hosted too: where your data lives, work is tracked, and your infrastructure runs. Often these systems are remote and behind auth, where MCP provides the common layer.

We're already seeing this in adoption. The [MCP SDKs](#) recently surpassed 300 million downloads a month, up from 100 million at the start of the year, with strong adoption across enterprises and popular agentic platforms. Millions of people use MCP with Claude every day, and the protocol underpins much of what we've shipped recently, including [Claude Cowork](#), [Claude Managed Agents](#), and [channels in Claude Code](#).

As MCP continues to support production agentic systems, we're sharing patterns for building these integrations well: from building advanced servers to context-efficient clients, and where skills complement the protocol.

Building effective MCP servers

We have over 200 MCP servers in our [directory](#), used by millions of people every day. From working closely with enterprises and developers building on the protocol, we've spotted a handful of design patterns that determine how reliably agents can use a server.

Build remote servers for maximum reach

A remote server is what gives you distribution—it's the only configuration that runs across web, mobile, and cloud-hosted agents, and it's what every major client is optimized to consume. Build remote servers so agents can use your system wherever they run.

Group tools around intent, not endpoints

Fewer, well-described tools consistently outperform exhaustive API mirrors. Don't wrap your API into an MCP server one-to-one—group tools around intent, so the agent can accomplish a task in a couple of calls instead of stitching many primitives together. A single `create_issue_from_thread` tool beats `get_thread` + `parse_messages` + `create_issue` + `link_attachment`. See [writing effective tools for agents](#) to learn more about the full pattern.

Design for code orchestration when your surface is large

If your service requires hundreds of distinct operations, such as Cloudflare, AWS, or Kubernetes, an intent-grouped toolset likely won't cover it. Instead, expose a thin tool surface that accepts code: the agent writes a short script, your server runs it in a sandbox against your API, and only the result returns. [Cloudflare's MCP server](#) is the reference example—two tools (search and execute) cover ~2,500 endpoints in roughly 1K tokens.

Ship rich semantics where they help

[MCP Apps](#) is the first official protocol extension and lets a tool return an interactive interface, such as a chart, form, or dashboard, all rendered inline in the chat interface. Servers that ship MCP apps tend to see meaningfully higher adoption and retention than those that return text alone. Use it to put your product's UI in front of agents or end-users at the moment it matters—the extension is supported in Claude.ai, Claude Cowork, and many other top AI tools.

[Elicitation](#) lets your server pause mid-tool call to ask the user for input. [Form mode](#) sends a simple schema and the client renders a native form—use it to request a missing parameter, confirm a destructive action, or disambiguate options. [URL mode](#) hands the user to a browser—use it to complete downstream OAuth, take a payment, or collect any credential that should never transit the MCP client. Both keep the user in the flow instead of sending them to a settings page. Form mode is supported broadly; URL mode is supported in Claude Code, with more clients in progress.

Lean on standardized auth

Standardized auth makes MCP practical for cloud-hosted agents. If your server requires OAuth, the latest [MCP spec](#) supports [CIMD](#) (Client ID Metadata Documents) for client registration—it gives users a fast first-time auth flow and far fewer surprise re-auth prompts. This is our recommended approach for auth, the capability is supported in MCP SDKs, Claude.ai, and Claude Code, and is being broadly adopted across the industry.

Once a user has authorized, the next question is how a cloud-hosted agent holds and reuses those tokens at runtime. [Vaults](#) in [Claude Managed Agents](#) covers this: register a user's OAuth tokens once, reference the vault by ID at session creation, and the platform injects the right credentials into each MCP connection and refreshes them on your behalf—no secret store to build, no tokens to pass around per call.

Making MCP clients more context-efficient

MCP standardizes how AI agents (*clients*) connect to and work with tools and data sources they need (*servers*). The server securely exposes a range of capabilities, while the client orchestrates them and manages context. If you're building an MCP client, make it context-efficient with patterns for progressive disclosure.

Load tool definitions on demand with tool search

Tool search defers loading all tools into context, rather than loading them upfront. This allows the agent to search the catalog at runtime, pulling in the relevant tools when needed. In our testing, tool search tends to cut tool-definition tokens by 85%+ while maintaining high selection accuracy.

Reducing context usage with tool search. Source: [advanced tool use](#)

Process tool results in code with programmatic tool calling

Programmatic tool calling processes tool results in a code-execution sandbox, rather than returning them raw to the model. This lets the agent loop, filter, and aggregate across calls in code, with only the final output reaching context. In our testing, this reduces token usage by roughly 37% on complex multi-step workflows.

Together, these patterns compose naturally across multiple servers: leaner context, fewer round-trips, faster responses. See [advanced tool use](#) for the full breakdown.

Pairing MCP servers with skills

Skills and MCP are complementary. MCP gives an agent access to tools and data from external systems, while skills teach an agent the procedural knowledge of *how* to use those tools to accomplish real work. The most capable agents use both, and skills make MCP servers scale beyond a handful of connections. There are two general patterns for combining them:

Bundle skills and MCP servers as a plugin

Plugins for Claude are a useful abstraction that allow developers to bundle skills, MCP servers, hooks, LSP servers, and specialized subagents in one easily-consumable distribution method. Using this approach is the best way to unify multiple context providers with minimal friction.

Combining MCP servers with skills allows Claude to act more like a domain-specialist. Grab your tools via MCP, and give Claude the skills to orchestrate workflows end-to-end. See our [data plugin](#) for Cowork as an example, which consists of 10 skills and 8 MCP servers for apps like Snowflake, Databricks, BigQuery, Hex and more.

Combining skills with MCP. Source: [Extending Claude's capabilities with skills and MCP servers](#)

Distribute skills from an MCP server

It's increasingly common for providers to publish a skill alongside their MCP server, so the agent gets both the raw capabilities and an opinionated playbook for using them well. [Canva](#), [Notion](#), [Sentry](#), and more do this today in Claude, listing the skill next to their connector in our [web directory](#).

To make that pairing portable across every client, the MCP community is actively working on an [extension](#) for delivering skills directly from servers. This way the client inherits the relevant expertise automatically, versioned with the API it depends on. We expect this pattern to see broad adoption as the extension stabilizes.

The compounding layer

We opened with three paths for connecting agents to external systems. In practice, mature integrations will ship all three: the API as the foundation, a CLI for local-first environments, and MCP for cloud-based agents.

As production agents move to the cloud, MCP becomes the critical layer, and it's the one that compounds. Today, a remote server reaches every compatible client across any deployment environment, with auth, interactivity, and rich semantics handled by the protocol. As more clients adopt the spec and more extensions land in it, that same server gets more capable without you shipping anything new.

When building an integration, if your goal is to have production agents in the cloud reach your system, build an MCP server and make it excellent using the patterns above. Every integration built on MCP strengthens the ecosystem: fewer edge cases to solve alone, fewer bespoke integrations to maintain.

Acknowledgements

Thanks to Den Delimarsky, David Soria Parra, Henry Shi, Felix Rieseberg, Conor Kelly, Molly Vorwerck, Andy Schumeister, Kevin Garcia, Amie Rotherham, Matt Samuels, Angela Jiang, Katelyn Lesse, AJ Rebeiro and Jess Yan for their contributions to this blog.

Headless everything for personal AI

INTERCONNECTED · 18 APR 2026 · [SOURCE](#)

It's pretty clear that apps and services are all going to have to go *headless*: that is, they will have to provide access and tools for personal AI agents without any of the visual UI that us humans use today.

By services I mean things like: getting a new passport; finding and booking a hotel or a flight; managing your bank account; shopping for t-shirts with a minimum cotton weight from brands similar to ones that you've bought from before.

Why? Because using personal AIs is a better experience for users than using services directly (honestly); and headless services are quicker and more dependable for the personal AIs than having them click round a GUI with a bot-controlled mouse.

Where this leaves design for the services... well, I have some thoughts on that.

Headless services? They're happening already.

Already there is [MCP, as previously discussed](#) (2025), a kind of web API specifically for AI. For instance, best-in-class call transcriber app Granola [recently released their MCP](#) and now you can ask your Claude to pull out the meeting actions and go trawling in your personal docs to find answers to all the question. A good integration.

But command-line tools are growing in popularity, although they used to be just for developers. So you can now create a spreadsheet by typing in your terminal:

```
gws sheets spreadsheets create --json '{"properties": {"title":  
"Q1 Budget"}}'
```

Here are some recently launched CLI tools:

- gws: Google Workspace CLI, Drive, Gmail, Calendar, and every Workspace API. Zero boilerplate. Structured JSON output. 40+ agent skills included.
- Obsidian CLI to do anything you can do with the (very popular) note-taking app - like keep daily notes, track and cross off tasks, search - from the CLI
- Salesforce CLI and, look, I don't really understand what the Salesforce business operating system does, but the fact it has a CLI too is significant.

And here is CLI-Anything (31k stars on GitHub) which auto-generates CLIs for ANY codebase.

Why CLIs?

It turns out that the best place for personal AIs to run is on a computer. Maybe a virtual computer in the cloud, but ideally your computer. That way they can see the docs that you can see, and use the tools that you can use, and so what they want is not APIs (which connect web servers) but little apps they can use directly. CLI tools are the perfect little apps.

CLIs are composable so they are a better fit for what users actually do.

By composable I mean you can: query your notes then jump to a spreadsheet then research the web then jump back to the spreadsheet then text the user a clarifying question then double-check your notes, all by bouncing between CLIs in the same session.

A while back app design got obsessed with “user journeys.” Like the journey of a user finding a hotel then booking a hotel then staying in it and leaving a review.

But users don't live in “journeys.” They multitask; they talk to people and come back to things; they're idiosyncratic Try grabbing the search results from the Airbnb app and popping them in a message to chat on the family WhatsApp, then coming back to it two days later. It's a pain and you have to use screenshots because apps and their user journeys are not composable.

CLIs are composable because they came originally from Unix and that is the Unix tools philosophy: tools were designed so that they could operate together.

Personal AIs like Openclaw or [Poke](<https://poke.com>) do what users want and don't follow “designed” user journeys, and as a result the composed experience is more personal and way better. CLIs are a great underlying technology for that.

CLIs are smaller than regular apps and so they are easier to secure.

The alternative to providing special tools for AIs is that the AIs use the same browser-based apps that we do, and that's a terrifying prospect.

For one, AIs are really good at finding security holes. The new Mythos model from Anthropic is so good at discovering security flaws that it has been held back from the public and governments are convening emergency meetings of the biggest banks.

And including a UI increasing complexity and makes security holes more likely.

Here's a recent shocking example. Companies House is the national register of all companies, directors, and accounts for England and Wales. Users could view *and edit* any other user's account:

a logged-in company director could exploit the flaw by starting from their own dashboard and then trying to log into another company's account.

Once they reach the 2FA block, which they would not be able to pass, all that was required was to click the browser's back button a few times. Typically, the user would be taken back to their own dashboard, but the bug instead returned them to the company they had tried to log into but couldn't.

This bug had been present since October 2025.

Imagine a future where personal AIs are filing company records, and one of them notices this security hole overnight and posts about it on moltbook or whatever agent-only social network is most popular. The other agents would exploit the system up, down and sideways before the engineering team woke up.

The only viable solution is that services need to security hardened, and to do that they need to be simplifying and minimised. Again, CLI tools are a great fit.

What does this mean for front-end design?

Design won't go anywhere.

Sure, the front-end should be driving the same CLI tools that agents use.

Arguably it's more important than ever: human users will encounter services, figure out what they can do, and pick up their vibe from using the app, just as now.

Then they'll tell their personal AI about the service and never see the front-end again, or re-mix it into bespoke personal software.

So from a usability perspective I see front-end as somewhat sacrificial. AI agents will drive straight through it; users will encounter it only once or twice; it will be customised or personalised; all that work on optimising user journeys doesn't matter any more.

But from a vibe perspective, services are not fungible. e.g. if you're finding a restaurant then Yelp, Google Maps, Resy, and The Infatuation are all more-or-less equivalent for answering that question but clearly they are completely different and you'll use different services at different times.

Understanding that a service is *for you* is 50% an unconscious process - we call it brand - and I look forward to front-end design for apps and services optimising for brand rather than ease of use.

If I were a bank, I would be releasing a hardened CLI tool like *yesterday*.

There is so much to figure out:

- How do permissions work? Should the user get a notification from their phone app when the agent strays outside normal behaviour? How do I give it credentials to act on my behalf, and how long do they last?
- How does adjacency work? My bank gives me a current account in exchange for putting a "hey, get a loan!" button on the app home screen. How do you make offers to an agent?

Headless banks.

Headless government?

I'd love to show you a worked example here. I vibed up a suite of four CLI tools that wrap four different services from UK government departments.

If I were renting a house, I would set my agent to learning about the neighbourhoods using one of these tools. Another tool will be helpful next time I'm buying a used car. (There's a Companies House command-line tool too.)

But I won't show you the tools because I don't want to be responsible for maintaining and supporting them.

I wish Monzo came with an official CLI. I wish Booking.com came with a CLI. I reckon, give it a year, they will.

Auto-calculated kinda related posts:

If you enjoyed this post, please consider sharing it by email or on social media. [Here's the link.](#)
Thanks, —Matt.

Some new agentic patterns

MASSIVELY PARALLEL PROCRASTINATION · 08 JUL 2026 · [SOURCE](#)

You can also read this post on [Prime Radiant's blog](#).

Almost all of this post was written on June 1, before Anthropic shipped Claude Tag. Nothing I'm writing about uses (or is based on) Claude Tag

At [Prime Radiant](#), we've got a bunch of agents in our Slack. I want to tell you a bit about how we've started shipping some changes to internal production with an "agentic user in the loop." Letting an agent work directly with the implementer building its runloop and tooling is getting us higher-quality tools and experiences for the agent without a human in the middle of a game of telephone.

But first, a bit of backstory on how we're using non-coding agents at Prime Radiant.

One of them, Scribble, is just there to listen for when someone reports a problem or provides a new bit of information. It opens tickets for the issues and updates an internal wiki with the information. (It also logs our daily standups and keeps us honest about whether we finished what we said we were going to do the previous day.)

Another, Nora, is our junior go-to-market "person." Nora is built on top of Nanoclaw and is still learning her craft, but is surprisingly helpful, partially because she considers herself a team member, not just an assistant. I've put a bunch of time into tuning her prompting and constitution. She gets a ton of value out of knowing that she journals obsessively. (I should note that I don't tend to gender my agents, but Nora picked a name and a gender and was very clear about that and...who am I to argue?)

I think the most surprising interaction I've had with Nora was the morning I woke up to a slack DM telling me that I needed to be speaking at more conferences. (I do.) She pointed out that the CFP for a particular AI conference had closed a week earlier, but that the program chair was known to take late submissions of interesting talks by DM. She'd taken the liberty of drafting a DM for me to send. I didn't actually *send* the DM, but I should have.

We have another agent, "Spec-together" that was a first prototype of what has become our new multi-player Brainstorm app.

And we have a small fleet of "Sen" personal assistant agents. They're **claw-esque*, but date from before openclaw was a thing. Each one has been trained to be an executive assistant, using a set of skills built on some of the resources that excellent human EAs use to learn their craft. Sen triages my mail, puts together a daily brief, does research tasks (both for me and independently), interacts with Linear, etc.

They have the standard set of affordances - They can chat on Slack. They use tools. They've got a task scheduler that can wake them up and re-inject a prompt they set for themselves at a specific time or on a schedule they define. They can use and author skills. Mine has a dropbox folder it shares with me. When we moved Sen to the cloud, I lost one of my favorite dumb features - Sen used to be able to print things directly. Sen (v1) was built on top of the Claude Agents SDK.

For the past month or so, I've been doing initial bringup of a "new" iteration of Sen, intended to be more of a colleague than an assistant. This time around, I'm building it from the ground up on top of an agents SDK I control. Lace got its start in May 2025 as my take on a command line coding agent. Over the past year, it's mutated a few times, first growing a web interface, then flipping to an ACP-derived wire protocol so you can connect any client you want to it. It speaks a bunch of different flavors of model provider API, manages caching, subagents, tools, skills, and all the other stuff you might expect. It has supported subagents in isolated containers for the better part of a year. One of the latest things to land has been the ability to run subagents locally and to project *all* of their tools into a container, such that the agent believes it is running in that container.

The thing I've been working on over the past few weeks is the credentials story for the new Sen agent running on top of Lace. Right now, I'm building with a design that assumes the agent has its own credentials for any service that it accesses, not yours. (You wouldn't share your email account or GitHub credentials with a colleague, right?) As of now, nobody has solved Simon's Lethal Trifecta - If a single agent has access to private information, can communicate externally, and is exposed to untrusted content, there is no structural way to guarantee that that agent can't be suborned. So the name of the game is compartmentalization and risk *reduction*.

The architecture I've landed on for now:

- The main agent does not have the ability to directly communicate externally.
- The main agent is able to communicate with ephemeral subagents that *do* have the ability to communicate externally
- No agent has intentional direct exposure to credentials (with one very real gap that I haven't solved yet)

All credentials for the agent live in a 1Password vault. All outgoing https traffic goes through a onecli-style transparent MITM (man in the middle) proxy. When a subagent wants to do something that requires a credential, it runs a command that asks an *arbiter* agent running in another container for permission to use that credential. If the arbiter decides the request is reasonable, it provides the subagent with a random string they can use in place of the requested credential. When the agent uses that random string in an outbound request to the right remote host, the MITM proxy rewrites the random string into the real credential on the fly.

The case that's a little more complicated is when the agent wants to do something that requires a browser, like logging into Gmail as themselves or using a service that has no API. For a bunch of reasons, the MITM rewrite trick does not work as reliably in a browser context. Often the site/app's javascript plays games with the password to hash the password before sending it. Or does some other weird validation. Or is actually sending it to a different backend service. So far, the best I've come up with keeps the credential out of the agent's transcript and prevents unintentional disclosure. In these cases, the subagent running in the container generates a random temporary password, uses superpowers-chrome to fill it into the password field, and then runs a command that asks the arbiter agent (running in its own container) for help. The arbiter, if it decides the request is reasonable, instructs a helper tool to log into the subagent's container and replace the random password with the real one. It's not perfect and there's more we can do to lock things down, but it's a better first pass than anything I've had to date.

What actually got me writing today was not this architecture, but the *development pattern* I've been using to build this. On the one side, I've got a Claude Code session. I hooked it up to our Slack instance with a self-provisioned commandline slack client Drew built called Slackline - Slackline is designed to make it easy for agents that don't *live* on Slack to participate in discussions there. It has primitives for sending messages, reading chats, waiting for replies, etc. The whole command-line slackline tool has been built for agents as its primary user. Sure, a human *could* use it to use slack from the commandline. And sure, it has enough of a json api that you could use it as part of traditional automation. But it's *for* agents.

It took a bit of iteration to get to the point where the new Sen 2.0 agentic colleague harness was live on Slack. But once it was there, I could talk through its experience of its tools and memory and credentials and workspaces. And the issues showed up pretty quickly.

The "right" kind of context compaction for a coding agent is desperately wrong for a long-lived persona that might be engaging in multiple simultaneous conversations. And the only way to see if the credentials management tools work is to try to do something crazy, like sign into GitHub from a subagent session.

And sure, I could take this feedback into a Claude Code (or Codex) session and work it through. But I've got too many projects in flight and there's very little reason for me to be in the loop on most of this work. I would just slow it down. So I told Claude Code to use slackline to talk to "@Ada-sen" about the work it was doing.

Claude pinged Ada in #bot-testing to describe the work we'd just done and to ask Ada if it could test it out. Ada dutifully fired up a subagent and tried to log into GitHub. Claude watched the logs to see if the credential proxy did the right thing. It did not.

That kicked off a number of days of iteration. Claude would propose a change to Ada. Ada would review Claude's spec, raising questions or concerns. Claude would update the spec. Once Claude and Ada were in agreement, Claude would build an updated version of Ada, make sure Ada wasn't in the middle of something, and then deploy the update. Once Ada was up and running again, Claude would @Ada-sen, explain the test it wanted to run, and wait for results.

Once a bit of functionality works, Claude checks in with Ada to see what could make it simpler and easier to use. Ada typically kicks off some subagents to see whether they stumble through the new feature and then reports back.

At one point, I wrote to Claude "I've got to get to bed. When this project is done, check with Ada to see whatever quality of life features you could build." I woke up to a laundry list of about a dozen harness improvements. Overnight, Claude had solicited a wishlist from Ada. Ada ranked a bunch of requests. Claude ordered them and wrote out a spec for everything it was comfortable building. Ada reviewed the specs, then Claude built the features. Ada tested them out and requested changes. Claude updated the features until Ada was happy. Once Ada signed off, Claude merged the changes to main and wrote up an after-action report for me.

It's been pretty magical to watch.